

Упражнение 7 и 8

Практически аспекти на алгоритъма RSA. Изчисляване на параметри. Бързо криптиране и декриптиране с използване на публичен и частен ключ.

1. Въведение

През 1975 г. трима изследователи в MIT са разработили алгоритъм за реализация на криптирането с публичен ключ - системата RSA, която носи имената на тримата изобретатели, а именно Рон Ривест, Али Шамир и Леонард Аделман.

Алгоритъмът RSA генерира първоначално два различни ключа за всеки потребител. Единият от тези два ключа се определя като публичен. Последният може да се разпространява свободно. Публичният ключ не може да се използва за декриптиране на съобщение - той може да се използва само за криптиране на съобщения изпратени до собственика на ключа. Само този, който притежава другия ключ, така наречения секретен (персонален) ключ, може да декриптира съобщенията, които са криптирани с публичния ключ.

Безспорно много математици са се опитвали да блокират алгоритъма на публичния ключ, като са правили различни изчисления, но до момента никой не е намерил алгоритъм, който да реши математическия проблем. Програмите за декриптиране се опитват да „разрушат“ ключа, но до момента това не е направено. Въпреки, че всичко това не е невъзможно, от гледна точка на изчисления, то е неуместно, тъй като публичният ключ е прекалено дълъг.

В повечето случаи за криптиране на съобщения не се използва RSA, защото изчисленията отнемат много дълго време. За повечето съобщения продължителното време за криптиране и декриптиране е неприемливо. RSA се използва за криптиране на симетричен ключ, който криптира самото съобщение. Стандартът SSL, който се използва за криптиране на web страници (URLs използва <https://> вместо <http://>) използва именно това свойство. Ключът се генерира в web браузъра и след това се изпраща към web сървъра. В случай, че не се използва криптирането с публичен ключ, ключа трябва да се изпраща през Интернет без да е защитен. За да се направи предаването на ключа сигурно, web сървъра изпраща своя публичен ключ към web браузъра. Последният избира симетричен ключ и криптира съобщението с публичния ключ на web сървъра и го връща обратно. Web сървърът е единствения, който може да декриптира публичния ключ със своя персонален ключ. RSA ключът се използва за симетричния ключ. От този момент нататък криптирането се прави със симетричен ключ, тъй като то е по-бързо от криптирането с публичен ключ. При тази система симетричните ключове могат да се избират произволно. В случай, че някой може да декодира едно криптирано съобщение, няма да получи никаква информация за ключовете, използвани

при останалите съобщения. В случай, че решите да криптирате електронни пощи, можете да криптирате симетричния ключ няколко пъти с различни публични ключове. Всеки публичен ключ принадлежи на един получател. При това положение всеки получател може да декриптира съобщението. Всеки публичен ключ формира съобщение, което съдържа същия ключ за декриптиране на оригиналното съобщение. Тази парадигма за гарантиране на защита се използва, например при PGP и по подобен начин се използва при SSL криптирането при web.

2. Приложение на RSA алгоритъм.

RSA е използван широко в много продукти, платформи и промишлености по света. Той намира разпространение в много търговски софтуерни продукти и е планирано да бъде използван в много повече приложения.

RSA е приложен в операционни системи на Microsoft, Apple, Sun и Novell. В практиката RSA може да бъде открит в телефони, на мрежови карти за Интернет и на смарт карти. Като цяло, RSA е внедрен като мярка за сигурност във всички главни протоколи за сигурни интернет-комуникации, включвайки S / MIME формат, SSL, и S / WAN.

RSA е също така използван вътрешно в много институции, включително клонове на американското правителство, водещи корпорации, национални лаборатории и университети.

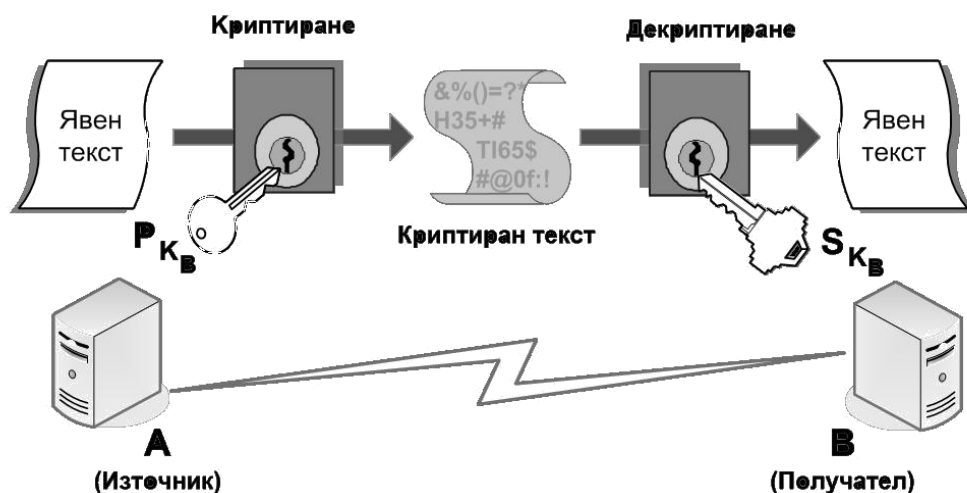
Лицензът за технологията на RSA е придобит от около 350 компании. Приблизителният брой на машините за кодиране на базата на RSA е около 300 милиона, правейки го най-широко използвания алгоритъм за криптиране в света.

В момента RSA се използва като част от SWIFT (Society for Worldwide Interbank Financial Telecommunications) – организация за световна комуникация между банки и финансови организации като стандарт за международен финансов трансфер и е възприет от ANSI X9.31 стандарта за банковата индустрия.

3. Изчисляване на параметри на RSA алгоритъм.

RSA е класически асиметричен алгоритъм, чиято сигурност се определя от факторизацията на големи прости числа. Сигурността на алгоритъма се базира на следните предпоставки:

- много е трудно да се подложи на криптоанализ основата на алгоритъма - факторизацията на големите числа;
- сигурността се основава на притежаването на секретен ключ единствено от едно лице - неговия собственик;
- секретният ключ не се предава никога по какъвто и да е било канал.



Алгоритъм за генериране на двойка ключове

1. Избират се две различни прости числа p и q . За по-добра сигурност те трябва да бъдат случайни и с еднакъв брой цифри (представени в двоична бройна система).
2. Изчислява се $n = pq$. Това е модулет на частния и публичния ключ.
3. Изчислява се функцията на Ойлер: $\Phi(n) = (p-1)(q-1)$.
4. Избира се цяло число e , така че $1 < e < \Phi(n)$, и освен това e и $\Phi(n)$ да нямат общи делители. Експонента на публичния ключ е e .
5. Изчислява се d , което удовлетворява уравнението $d * e \equiv 1 \pmod{\Phi(n)}$. С други думи, $e * d - 1$ може да бъде разделено без остатък на $(p-1) * (q-1)$. С други думи $d = t \pmod{\Phi(n)}$, където t е параметър изчислен от разширения Евклидов алгоритъм). Експонента на частния ключ е d .

Публичният ключ е съставен от модула n и от публичната експонента e . Частният ключ съдържа модула n и частната експонента d .

Пример 1:

Алиса иска да изпрати шифровано съобщение до Боб. Боб първо изчислява своите RSA параметри в Стъпки 1–5. След това той изпраща на Алиса неговия публичен ключ. Алиса криптира съобщението ($x = 4$) и изпраща шифротекста y на Боб. Боб дешифрира y , използвайки личния си ключ.

Алис

Изпраща съобщение $x = 4$

Боб

1. избира прости числа $p = 3$ и $q = 11$
2. изчислява $n = p \cdot q = 33$
3. изчислява функцията: $\Phi(n) = (3-1)(11-1) = 20$
4. избира цяло число $e = 3$
5. изчислява $d \equiv e^{-1} \equiv 7 \pmod{20}$

Боб връща публичния ключ на Алис

$$k_{pub} = (33, 3)$$

← -----

Алис

Алис връща шифротекста на Боб

$$y = x^e \equiv 4^3 \equiv 31 \pmod{33}$$

----- →

$$y = 31$$

Боб

Боб разшифрова текста, използвайки личния си ключ.

$$y^d = 31^7 \equiv 4 = x \pmod{33}$$

Обърнете внимание, че частните и публичните компоненти изпълняват условието:

$$e \cdot d = 3 \cdot 7 \equiv 1 \pmod{\Phi(n)}.$$

Практическите параметри на RSA са много, много по-големи. n трябва да бъде дълъг поне 1024 бита, което води до дължина за p и q от 512. Пример за RSA параметри за тази битова дължина:

$p = E0DFD2C2A288ACEBC705EFAB30E4447541A8C5A47A37185C5A9CB98389CE4DE19199AA3069B404FD98C801568CB9170EB712BF10B4955CE9C9DC8CE6855C6123h$

$q = EBE0FCF21866FD9A9F0D72F7994875A8D92E67AEE4B515136B2A778A8048B149828AEA30BD0BA34B977982A3D42168F594CA99F3981DDABFAB2369F229640115h$

$n = CF33188211FD6052BDBB1A37235E0ABB5978A45C71FD381A91AD12FC76DA0544C47568AC83D855D47CA8D8A779579AB72E635D0B0AAAC22D28341E998E90F82122A2C06090F43A37E0203C2B72E401FD06890EC8EAD4F07E686E906F01B2468AE7B30CBD670255C1FEDE1A2762CF4392C0759499CC0ABECFF008728D9A11ADF_h$

$e = 40B028E1E4CCF07537643101FF72444A0BE1D7682F1EDB553E3AB4F6DD8293CA1945DB12D796AE9244D60565C2EB692A89B8881D58D278562ED60066DD8211E67315CF89857167206120405B08B54D10D4EC4ED4253C75FA74098FE3F7FB751FF5121353C554391E114C85B56A9725E9BD5685D6C9C7EED8EE442366353DC39_h$

$d = C21A93EE751A8D4FBFD77285D79D6768C58EBF283743D2889A395F266C78F4A28E86F545960C2CE01EB8AD5246905163B28D0B8BAABB959CC03F4EC499186168AE9ED6D88058898907E61C7CCCC584D65D801CFE32DFC983707F87F5AA6AE4B9E77B9CE630E2C0DF05841B5E4984D059A35D7270D500514891F7B77B804BED81_h$

4. Шифриране и дешифриране - бързо експониране .

За разлика от симетричните алгоритми като AES, DES или поточни шифри, алгоритмите с публичен ключ се основават на аритметика с много дълги числа. RSA криптиране и декриптиране се основават на модулна експозиция:

- криптиране

- декриптиране

o

Правия начин на експониране изглежда така:

SQ MUL MUL MUL

където SQ означава квадратиране, а MUL умножение. Експонентите e и d като цяло са много големи. Експонентите обикновено се избират в диапазона от 1024-3072 бита или дори по-големи. (Понякога публичният експонент e е избран като малка стойност, но d винаги е много дълъг.) Правото експониране, ще изисква около 21024 или повече умножения. Централният въпрос е дали има значително по-бързи методи за експоненциране. Отговорът е, за щастие, да. Иначе бихме могли да забравим за RSA и почти всички други криптосистеми с публичен ключ, които се използват днес, тъй като всички те разчитат на експониране. Един такъв метод е алгоритъмът за квадратите и умножението. Първо показваме няколко илюстративни примера с малки числа, преди да представим действителния алгоритъм.

Пример 2:

Нека разгледаме колко умножения са необходими за изчисляване на простата експоненция x^8 . С директния метод:

$$x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{MUL} x^4 \xrightarrow{MUL} x^5 \xrightarrow{MUL} x^6 \xrightarrow{MUL} x^7 \xrightarrow{MUL} x^8$$

Необходими са седем умножения. Като алтернатива може да се направи нещо по-бързо:

$$x \xrightarrow{SQ} x^2 \xrightarrow{SQ} x^4 \xrightarrow{SQ} x^8$$

което изисква само три квадратури, които са приблизително толкова сложни, колкото и умножението.

Този бърз метод работи добре, но е ограничен до експоненти, които са с мощност 2, т.е. стойности e и d от формата 2^i .

Пример 3:

Изчисляване на експонентата на x^{26} . Правия метод ще изисква 25 умножения. По-бързия начин е следният:

$$x \xrightarrow{SQ} x^2 \xrightarrow{MUL} x^3 \xrightarrow{SQ} x^6 \xrightarrow{SQ} x^{12} \xrightarrow{MUL} x^{13} \xrightarrow{SQ} x^{26}.$$

Този подход отнема общо шест операции, две умножения и четири квадратираия.

Разглеждайки последния пример, виждаме, че може да се постигне желания резултат, като се извършат две основни операции:

1. изравняване на текущия резултат,
2. умножаване на текущия резултат от основния елемент x .

В горния пример се изчислява последователността SQ, MUL, SQ, SQ, MUL, SQ.

Проблемът е, че не се знае последователността, в която трябва да се извършат операциите. Едно от решенията на проблема е да се използва алгоритъмът *square-and-multiply*. Той осигурява систематичен начин за намиране на последователността, в която трябва да се извършат операциите по x за изчисляване на x^N . Алгоритъмът работи по следния начин:

Алгоритъмът се основава на сканиране на бита на експонента от ляво (най-значимият бит) на дясно (най-малко значимият бит). Във всяка итерация, т.е. за всеки бит на експонентата, текущият резултат е квадрат. Ако текущо сканираният бит на експонентата има стойност 1, след уравниването се извършва умножение на текущия резултат по x .

Пример 4:

Отново разглеждаме експоненцията x^{26} . За алгоритъма square-and-multiply бинарното представяне на експонентата е решаващо:

Алгоритъмът сканира битовете, като се започне отляво с h_4 и завършва с най-десния бит h_0 .

Стъпка		
0		начална настройка, битова обработка: $h_4 = 1$
1a	()	SQ, битова обработка: h_3
1a		MUL, тъй като $h_3 = 1$
2a	() ()	SQ, битова обработка: h_2
2b		няма MUL, тъй като $h_2 = 0$
3a	() ()	SQ, битова обработка: h_1
3b		MUL, тъй като $h_1 = 1$
4a	() ()	SQ, битова обработка: h_0
4b		няма MUL, тъй като $h_0 = 0$

Първата основна операция, квадратиране, води до смяна отляво на експонента, като 0 е поставена в най-дясната позиция. Другата основна операция, умножението по x , води до запълване на 1 в най-дясното положение на експонента.

5. Сравнение между криптирането със секретен и публичен ключ

Основното предимство на криптирането с публичен ключ пред това със секретен ключ е, че персоналните ключове никога не се предават. Това прави този тип криптография по-сигурна и удобна. В една система със секретен ключ, е необходимо предаване на ключовете, което е свързано с рискове. Освен това при работа със секретни ключове механизма на идентифициране се осъществява трудно. Когато един цифров подпис използва инфраструктура с публичен ключ се налага предаване на секретна информация. За да се избегне отказ на плащане се налага трета страна да проверява идентичността.

Безспорно криптирането с публичен ключ има няколко недостатъка. Повечето технологии на секретни ключове са по-бързи от алгоритмите за криптиране с публичен ключ. Тъй като бързината е по-голяма с порядъци, криптирането с публичен ключ не е препоръчително да се използва за големи файлове. За да може една система да бъде и защитена и бърза, е необходимо да се комбинират и двата типа криптография.

При такава комбинация съобщението ще се криптира със секретен ключ, защото при криптиране с публичен ключ ще се отнеме много време, а секретния ключ се

прикрепва към съобщението, като самия секретен ключ е криптиран с публичен ключ. По този начин се постига и по-висока скорост и защита.

При SSL криптирането, което се използва за сигурен обмен на информация през web, криптирането с публичен ключ се използва за размяна на секретния ключ. Web сървърът изпраща своя публичен ключ към web браузъра. Последният създава ключ на сесия и криптира ключа на сесия с публичения ключ на web сървъра. След това ключа на сесия се предава обратно на web сървъра, който го декриптира с помощта на своя секретен ключ. По този начин ключовете на сесии могат спокойно да се предават през незащитени мрежи. След като ключа на сесия се предаде, той се използва за криптиране на връзката, тъй като е доста по-бърз. Алгоритмите за секретните ключове ще бъдат от значение до момента, в който компютрите не станат поне хиляди пъти по-бързи от съвременните компютри. Ключът на сесия е сигурен, защото е валиден само за една определена сесия и след това не може да се използва повече.

Задачи за самостоятелна работа:

Задача 1. Използвайки алгоритъм *square-and-multiply* намерете експонентата на x^{37} . Колко стъпки са необходими за преобразуването?

Задача 2. Използвайки алгоритъм *square-and-multiply* намерете експонентата на x^{71} . Колко стъпки са необходими за преобразуването?

Задача 3. Да се напише програма в CrypTool, която генерира ключове за RSA алгоритъм. Използвайте RSA Key Generator.

Задача 4. Да се напише програма в CrypTool, която шифрира текста “RSA е алгоритъм за криптография с публичен ключ. Това е първият алгоритъм, който е подходящ за цифрово подписване криптиране. RSA се използва широко в протоколите за електронна търговия и се смята, че е достатъчно сигурен при достатъчно дълги ключове и използването на актуални реализации.”.



- Тествайте задачата с дължина на параметрите p и $q = 20$ bits и публична експонента = 11.

- Тествайте задачата с дължина на параметрите p и $q = 30$ bits и публична експонента $= 7$.

Какви резултати се получават? Каква е зависимостта между параметрите?

Задача 5. Да се напише програма в C# в СгурTool, която дешифрира текста “RSA е алгоритъм за криптография с публичен ключ. Това е първият алгоритъм, който е подходящ за цифрово подписване криптиране. RSA се използва широко в протоколите за електронна търговия и се смята, че е достатъчно сигурен при достатъчно дълги ключове и използването на актуални реализации.”.



- Тествайте задачата с параметрите от задача 4. Какви резултати се получават?
- Разменете ключовете и направете сравнение на резултатите. Каква е зависимостта между параметрите?